A Binary Stack-Alphabet Hierarchy for Deterministic Pushdown Automata with ε-Moves

Completion Slopes, Up-Modes, and a Product Lower Bound

Alp Eren Bütün
Independent Researcher
GitHub: github.com/alperenbutun

August 2026

Abstract

For fixed numbers of control states and pushdown symbols, the descriptional power of deterministic pushdown automata depends on how information is divided between finite control and the stack alphabet. Masopust established a binary stack-alphabet hierarchy for stateless deterministic pushdown automata and extended the hierarchy to m-state *realtime* deterministic pushdown automata, while leaving the corresponding question for general m-state deterministic pushdown automata with ε-moves open. We answer that question affirmatively for every fixed state budget $m \geq 2$ in Masopust's general single-initial-symbol model.

For every $m \geq 2$ and $k \geq 1$ we construct a binary language $B_{m,k}$ recognized by a realtime DPDA with exactly m states and $k+1$ pushdown symbols. The construction contains $mk+1$ distinguished state/top-symbol modes whose unary completion lengths have pairwise distinct affine slopes. We prove a representation-independent lower bound: every DPDA in the same general model, even with arbitrary deterministic ε-moves, that recognizes $B_{m,k}$ satisfies

$$\boxed{|Q|\,|\Gamma| \geq mk + 1.}$$

The proof uses four ingredients. First, distinct completion lengths prevent repetition of full configurations along each live accumulation ray. Second, escape of the raw stack height yields a context-independent height-increasing pump between two suffix-minimum positions. Third, one such *up-mode* cannot support two different completion slopes. Finally, $mk+1$ slopes therefore force $mk+1$ distinct state/top-symbol modes.

Consequently, if $\mathcal{D}_{m,k}$ denotes the languages recognized by DPDAs with at most m states and at most k pushdown symbols in the general single-initial-symbol model, then

$$\boxed{\mathcal{D}_{m,k} \subsetneq \mathcal{D}_{m,k+1} \qquad (m \geq 2,\ k \geq 1)}$$

already over the binary input alphabet. The paper also records the development of the witness from earlier four-letter and three-letter completion-slope constructions, and explains how the lower-bound viewpoint grew out of completion-aware state–stack tradeoffs observed in earlier counting DPDAs.

Keywords: deterministic pushdown automata; stack alphabet; descriptional complexity; hierarchy; epsilon transitions; lower bounds.

1 Introduction

The number of pushdown symbols is a genuine descriptional resource. A larger stack alphabet may move finite information out of the control state set, while fewer stack symbols can force the control to remember distinctions that would otherwise be visible at the top of the stack. This tradeoff is classical, and general transformations between state and stack-symbol resources have been studied for decades [2, 3]. What is substantially more delicate is an *exact fixed-state hierarchy*: for a fixed state budget m, does allowing one additional stack symbol strictly increase the class of languages recognized by deterministic pushdown automata?

Masopust proved such an infinite hierarchy for stateless deterministic pushdown automata over a binary alphabet and then for m-state *realtime* deterministic pushdown automata for every $m \geq 1$ [1]. His realtime witnesses do not automatically settle the model with ε-moves: indeed, the paper exhibits a small general DPDA using ε-moves that collapses the intended realtime separation mechanism. The existence of a corresponding hierarchy for general m-state deterministic pushdown automata was therefore left open [1]. The main result of the present paper resolves this question for every $m \geq 2$, while retaining a binary witness alphabet.

1.1 The open hierarchy problem

The distinction between the realtime and general deterministic models is the central external problem addressed in this paper. In a realtime DPDA every transition consumes an input symbol, so the evolution of the finite control and the visible top of the stack is synchronized with the input. A general DPDA may instead perform a deterministic block of ε-moves between two input symbols. Such a block can rewrite the stack, move information between finite control and the store, or perform internal cleanup before the next input symbol is seen. Consequently, a witness that separates two realtime resource classes may cease to separate the corresponding general classes.

Masopust's paper makes this obstruction explicit: after proving the binary hierarchy for m-state realtime DPDAs, it leaves open whether an analogous hierarchy exists for general m-state deterministic pushdown automata [1]. In the notation fixed formally in section 3, the question is the following.

Open Problem 1.1 (Masopust's fixed-state stack-alphabet hierarchy question)**.** For a fixed state budget m, does increasing the available pushdown alphabet from k to $k+1$ strictly increase the class of languages recognized by general deterministic pushdown automata, even when deterministic ε-moves are allowed? Can such a separation be witnessed over a fixed, in particular binary, input alphabet?

The difficulty is therefore not merely to build an $(m, k+1)$-DPDA. One must prove that *every* equivalent DPDA with at most m states and at most k stack symbols fails, including machines that use ε-moves and completely different stack encodings from the upper-bound construction. This is why a representation-scoped counting argument is insufficient for the present problem.

Resolution of the open question. For the general single-initial-symbol model and every $m \geq 2$, $k \geq 1$, we construct a binary witness $B_{m,k}$ and prove

$$B_{m,k} \in \mathcal{D}_{m,k+1} \qquad \text{but} \qquad B_{m,k} \notin \mathcal{D}_{m,k}.$$

In fact the proof gives the stronger product lower bound

$$|Q|\,|\Gamma| \geq mk + 1$$

for every general DPDA recognizing the witness. Thus the open hierarchy question is answered by a quantitative mode-counting theorem rather than by a restriction on the opponent's internal representation. The separately formulated stateless case $m = 1$ is already covered by Masopust under his stateless initial-store convention; section 3 records the convention difference explicitly.

This paper gives a binary witness family for the general model when $m \geq 2$. The main quantitative statement is stronger than the adjacent separation needed for the hierarchy. For each pair (m, k) we define a binary language $B_{m,k}$ such that every DPDA recognizing it obeys

$$|Q|\,|\Gamma| \geq mk + 1. \tag{1}$$

On the other hand $B_{m,k}$ has an explicit realtime recognizer with m states and $k + 1$ stack symbols. Thus an m-state recognizer with only k stack symbols is impossible.

The lower bound is not tied to any prescribed stack representation. This point is central. Earlier work by the present author arose from a concrete completion-aware DPDA for a binary counting language and led to exact, but representation-scoped, state–stack tradeoffs [7, 8]. Those results could constrain a chosen residual encoding, but not an arbitrary recognizer that might encode the same semantic information in a completely different way. The present proof removes that representation assumption. It asks only which words must be accepted after certain prefixes and derives from those right-continuation requirements a lower bound on the number of state/top-symbol modes available to *every* deterministic recognizer.

Completion slopes. The witness is organized around $mk + 1$ families of live prefixes. On family s, after n accumulation symbols have been read, the unique all-zero completion has length

$$N_s(n) = \ell - s + (s + 1)n, \qquad \ell = mk + 1.$$

Hence the $mk + 1$ families expose the pairwise distinct slopes

$$1, 2, \ldots, mk + 1.$$

The central proof principle is that one recurrent pushdown mode cannot realize two distinct slopes. Once this is established, the product bound follows immediately because a DPDA has at most $|Q||\Gamma|$ nonempty modes.

Contributions. The paper has five principal contributions.

1. We give an explicit binary realtime DPDA $U_{m,k}$ with m states and $k + 1$ stack symbols and define the witness language $B_{m,k} = L(U_{m,k})$.

2. We isolate an exact regular-shaped slice of $B_{m,k}$: for $0 \leq s < mk + 1$,

$$10^s1^n0^r \in B_{m,k} \quad \Longleftrightarrow \quad r = mk + 1 - s + (s + 1)n.$$

3. We prove an ε-robust *step-pump lemma*: along any live ray whose raw stack height escapes every bound, two suffix-minimum positions with the same state/top mode yield a context-independent, positive-input, height-increasing pump.

4. We prove the *slope-separation lemma*: two distinct affine completion slopes cannot share one up-mode in any DPDA recognizing the witness.

5. We obtain the unrestricted product lower bound (1) and the binary adjacent stack-alphabet hierarchy for every $m \geq 2$ and $k \geq 1$.

Organization. Section 2 records the completion-aware DPDA lineage and the four-letter to three-letter to binary witness development. Section 3 fixes the exact machine model. Section 4 defines the binary witness machine and proves the exact completion slice. Sections 5 to 7 develop the lower-bound machinery. Section 8 proves the product theorem and hierarchy corollary. Section 10 places the result relative to earlier state/stack-symbol work and discusses scope, while Section 11 records remaining questions.

2 From the 2023 DPDA to representation-independent hierarchy witnesses

The hierarchy proof is self-contained and does not depend formally on the counting automata in this section. The purpose of the section is to record the mathematical route by which a concrete completion-aware stack design led first to more compact residual encodings and then to the representation-independent continuation-slope invariant used in the main lower bound.

2.1 The original 2023 completion-aware DPDAs

The project began with the binary counting language

$$L_{2,1} = \{w \in \{0,1\}^* : N_1(w) = 2N_0(w)\}.$$

A concrete DPDA developed by the author in 2023 used a center/left/right organization and a stack whose operational meaning was "what is still needed" to restore the 2:1 balance. The later completion-frontier paper retained two formal variants of this original construction: one excludes the empty input and one accepts it. We reproduce those same formal redrawings in figures 1 and 2. They are included for lineage and motivation; the hierarchy theorem proved later is independent of their transition structure [7].

The notation $a, A/\gamma$ means that input a is consumed, top symbol A is replaced by γ, and the indicated control transition is taken. The dashed boxes are not ordinary ε-moves. They list completion operations enabled only after the binary input is exhausted, represented by the right endmarker $\dashv$. This separation preserves ordinary DPDA determinism.

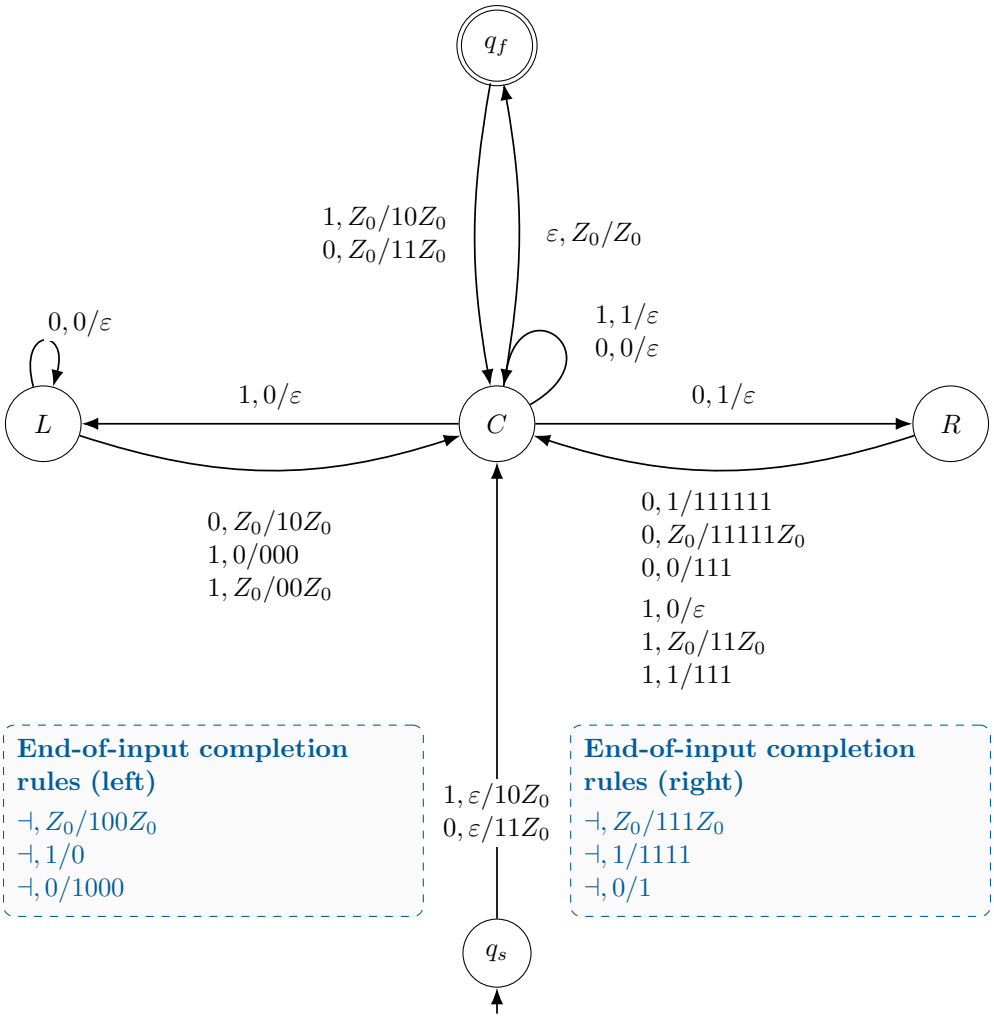

Figure 1: Formal redrawing, as used in the first companion paper, of the original 2023 2:1 DPDA variant with the empty input excluded. Solid arcs are ordinary recognition transitions; dashed callouts are right-endmarker completion operations.

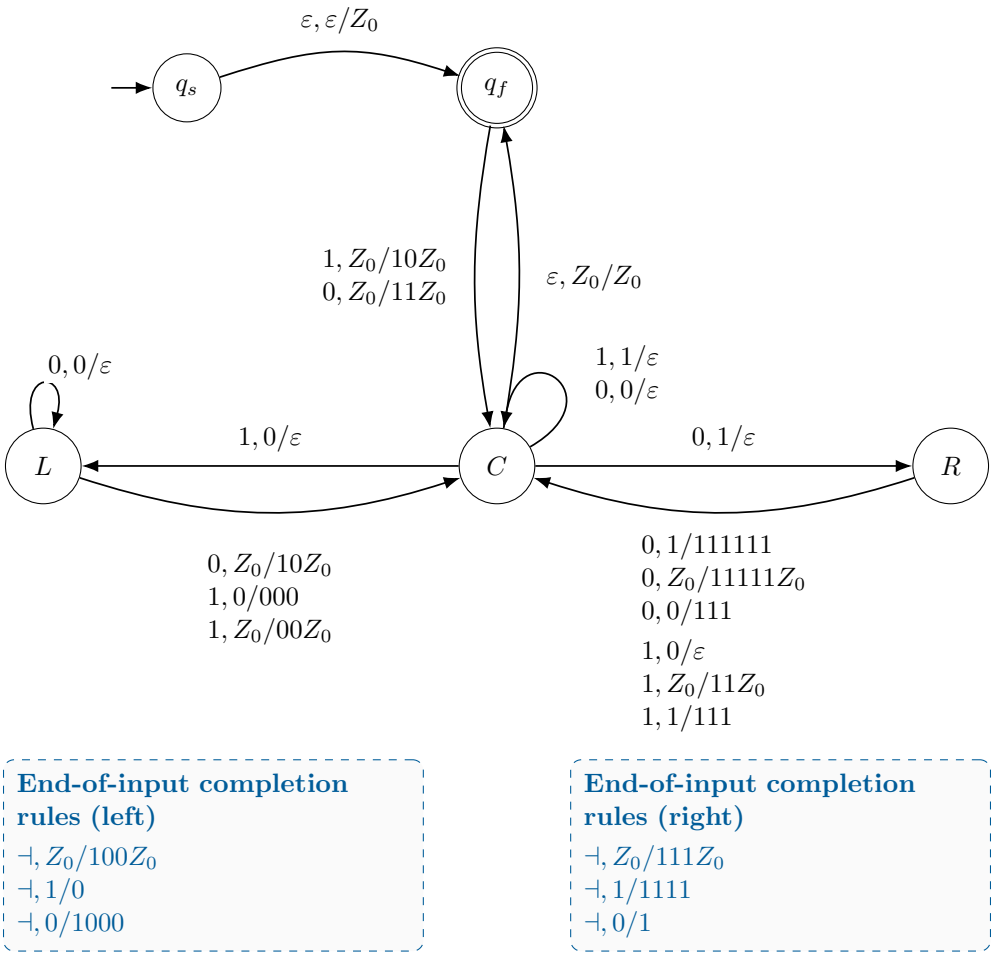

Figure 2: The empty-input-included formal variant used in the first companion paper. The center/left/right recognition logic and completion callouts are unchanged; the initialization wrapper additionally accepts ε, so the standard language $L_{2,1}$ is recognized.

The important feature for the later work was not merely recognition. The state and stack jointly encoded a signed completion demand, while the end-of-input operation exposed a shortest continuation. This observation eventually led to residual, quotient, geodesic, and state–stack resource analyses in the two companion manuscripts [7, 8].

2.2 The later resource-improved $L_{2,1}$ DPDA

The second companion paper subsequently changed the residual coordinates themselves. Instead of retaining a representation whose sign changes require normalization, it aligned the finite residual bases with the sign-changing pop. For general $L_{p,q}$ this gives a boundary-compatible stack alphabet

$$\{0, 1, Z_0\}$$

with $\max(p, q)$ residual-control states and no ε-normalization: each binary residual edge is realized by one ordinary input transition [8]. For $p = 2, q = 1$, the residual control therefore consists of the two states R_0, R_1. The right-endmarker acceptance wrapper is shown separately in figure 3.

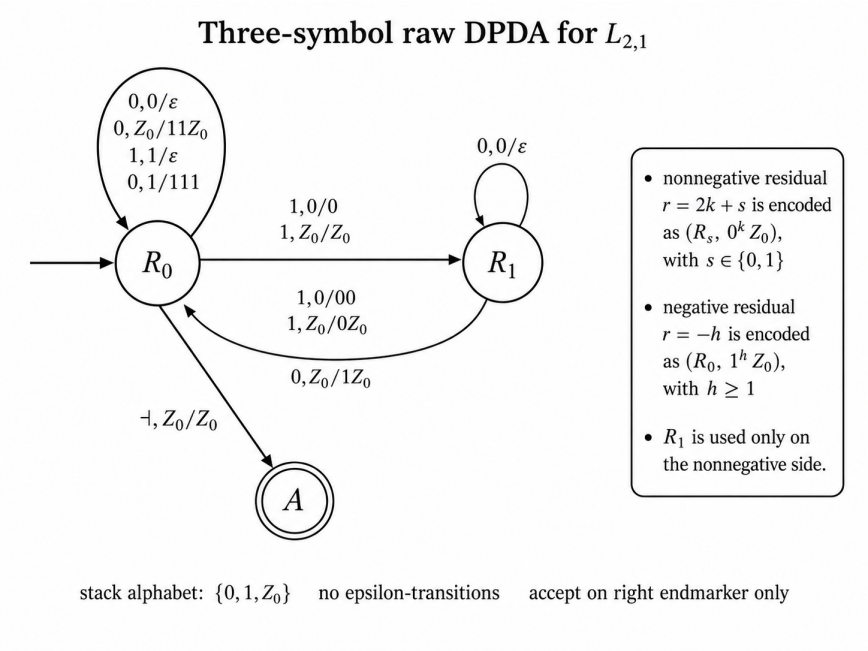

Figure 3: Later three-symbol raw DPDA for $L_{2,1}$ from the second companion study. The stack alphabet is $\{0, 1, Z_0\}$; the residual control uses R_0, R_1, there are no ε-normalization moves, and the right endmarker handles acceptance. This construction is more resource-efficient than the original center/left/right realization for the residual-conjugacy objective, but no unrestricted global optimality claim for all recognizers is needed here.

The improvement is easiest to see by separating the resources that are actually being compared. The original machine was designed to make completion behavior visible and uses the center/left/right control logic together with an initialization/acceptance wrapper. The later machine was designed for raw residual–configuration conjugacy. For $L_{2,1}$ it keeps the same three-symbol physical stack alphabet but compresses the residual-control component to R_0, R_1 and removes ε-normalization from ordinary binary processing.

Construction	Main purpose	Stack alphabet	Control/normalization feature
Original 2023 DPDA	recognition plus explicit end-of-input completion behavior	$\{0, 1, Z_0\}$	center/left/right semantic core plus wrapper; completion is a separate end-of-input phase
Later boundary-compatible DPDA	raw residual–configuration conjugacy	$\{0, 1, Z_0\}$	two residual-control states R_0, R_1 for $p = 2, q = 1$; no ε-normalization on binary residual edges

Table 1: What is improved in the later $L_{2,1}$ construction. The point is not a claimed global state minimum for language recognition; it is a more compact and more literal residual realization under the stated conjugacy objective.

This distinction is important for the third paper. The first two studies taught us that the same semantic quantity can be redistributed between state and stack in substantially different ways. That observation suggests a descriptional question, but it does not itself prove an unrestricted lower bound: an adversarial recognizer is not required to use either of the two displayed coordinate systems.

This improvement sharpened the conceptual question. The earlier papers could prove exact lower bounds only after fixing substantial features of the residual representation. They therefore did *not* show that every possible DPDA recognizer had to pay the same resource cost. The present paper asks instead:

> Can the continuation language itself force a state–stack lower bound, without prescribing how an equivalent DPDA represents the hidden information?

The completion-slope method below answers this by turning right-continuation lengths into an externally visible quantity. The recognizer is free to choose any internal stack code.

2.3 From four letters to three, and then to binary

The first general prototype used semantic modes $(i, t) \in \mathbb{Z}_m \times \mathbb{Z}_k$ and a distinct weight

$$\omega(i, t) = tm + i + 1 \in \{1, \dots, mk\}.$$

Over four input letters, two control symbols moved independently through the two semantic coordinates, an accumulation symbol added the current weight, and a delimiter started unary cleanup. This yielded mk distinct completion slopes plus one separate cleanup role.

The next version observed that the two coordinates could be traversed by one mixed-radix control symbol. The same mechanism therefore required only three input letters. The remaining third symbol was still used as a delimiter, and a direct binary coding would have introduced phase-decoding overhead that was incompatible with the exact m-state upper budget.

The decisive binary step was not an encoding of the three-letter witness. Instead we changed the invariant: rather than requiring

$$mk \text{ accumulation slopes} + 1 \text{ cleanup mode},$$

we force directly

$$\boxed{mk + 1 \text{ pairwise distinct completion slopes.}}$$

That removes the need for a separate delimiter mode and yields the binary witness studied in the remainder of the paper. For reference, the development is summarized without introducing another automaton drawing:

Stage	Input alphabet	Lower-bound observable
Four-letter prototype	4 symbols	mk slopes + separate cleanup role
Mixed-radix prototype	3 symbols	same product mechanism with one control letter
Final witness $B_{m,k}$	$\{0,1\}$	$mk+1$ distinct completion slopes directly

Table 2: Evolution of the hierarchy witness. The intermediate constructions motivate the final invariant; only the binary witness is needed for the main theorem.

Why the binary construction is not just an encoding trick. A direct binary code for the three-letter prototype would require the recognizer to remember where it is inside a codeword or which phase a binary block represents. Under an exact m-state upper budget, that decoder memory is itself a descriptional resource and can destroy the intended comparison. The final construction therefore changes the mathematical observable rather than merely recoding letters. By replacing "mk slopes plus a special cleanup role" with $mk+1$ slopes, both selection and cleanup can be driven by the same binary alphabet. This is the conceptual reason the final proof is cleaner than the earlier prototypes.

What the intermediate prototypes contribute. The four-letter construction identified the product phenomenon: many externally distinguishable completion rates should force many state/top modes. The three-letter construction showed that two semantic coordinates do not require two separate control letters. The binary construction then removed the final phase marker altogether. These stages are included because they explain why the final witness has its particular shape; none of the intermediate languages is used as a premise in the proof of the binary theorem.

3 Model and notation

We use the general deterministic pushdown model of Masopust [1]. A pushdown automaton is a tuple

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F),$$

where Q, Σ, and Γ are finite, $q_0 \in Q$, $Z_0 \in \Gamma$, and $F \subseteq Q$. The transition function is partial and has the form

$$\delta : Q \times \Gamma \times (\Sigma \cup \{\varepsilon\}) \to Q \times \Gamma^*.$$

The stack top is written on the right. Thus if

$$\delta(q, X, a) = (p, \beta),$$

then

$$(q, \gamma X, av) \vdash_M (p, \gamma\beta, v).$$

No transition is available from the empty stack.

The machine is deterministic if for each (q, X) there is at most one transition on each possible input symbol and, whenever $\delta(q, X, \varepsilon)$ is defined, no input-consuming transition $\delta(q, X, a)$ with $a \in \Sigma$ is defined. A DPDA is *realtime* if it has no ε-transitions.

Acceptance is by final state and empty stack after all input is consumed:

$$L(M) = \{w : (q_0, Z_0, w) \vdash_M^* (f, \varepsilon, \varepsilon) \text{ for some } f \in F\}.$$

Definition 3.1 (Control-store configuration and mode)**.** A *control-store configuration* is a pair $(q, \gamma) \in Q \times \Gamma^*$. Its height is $\mathrm{ht}(q, \gamma) = |\gamma|$. If $\gamma = \eta X$ is nonempty, its *mode* is the state/top-symbol pair

$$[q, X].$$

Thus a DPDA has at most $|Q||\Gamma|$ nonempty modes. The terminology is standard in unary DPDA analysis; see, e.g., Pighizzini [5].

For integers $m, k \geq 1$, define

$$\mathcal{D}_{m,k} = \{L(M) : M \text{ is a DPDA in the above model, } |Q_M| \leq m,\ |\Gamma_M| \leq k\}.$$

The inclusion

$$\mathcal{D}_{m,k} \subseteq \mathcal{D}_{m,k+1}$$

is immediate, since an additional unused stack symbol may always be added. The problem is strictness.

Open Problem 3.2 (Formal form of Open Problem 1.1)**.** For fixed m and every $k \geq 1$, determine whether

$$\mathcal{D}_{m,k} \subsetneq \mathcal{D}_{m,k+1}$$

holds for general DPDAs with deterministic ε-moves, preferably over a fixed binary input alphabet.

The upper inclusion is automatic; all mathematical work is therefore in the strictness witness and its lower bound. Notice also that the lower-bound opponent may choose any transition structure, any stack code, and any deterministic ε-normalization compatible with the model. The proof below must survive all of those choices.

Remark 3.3 (The case $m = 1$)**.** The new construction below is stated for $m \geq 2$. Masopust's stateless hierarchy uses a closely related but separately stated stateless convention in which the initial pushdown content may be a nonempty word rather than a single initial symbol [1]. To avoid silently identifying two conventions, the present theorem treats the general single-initial-symbol model for $m \geq 2$ and cites the known stateless result separately.

4 The binary witness machine

Fix

$$m \geq 2, \qquad k \geq 1,$$

and put

$$\ell = mk + 1.$$

We now define a realtime DPDA $U_{m,k}$ over $\{0, 1\}$.

4.1 Slope modes

Let

$$Q = \{q_0, q_1, \ldots, q_{m-1}\}$$

and

$$\Gamma = \{A, M_0, M_1, \ldots, M_{k-1}\}.$$

Hence

$$|Q| = m, \qquad |\Gamma| = k + 1.$$

Define a permutation π of $\{0, \ldots, m-1\}$ by

$$\pi(0) = 1, \qquad \pi(1) = 0, \qquad \pi(r) = r \quad (2 \le r < m).$$

We choose ℓ pairwise distinct slope modes

$$x_s = (r_s, X_s), \qquad 0 \le s < \ell,$$

as follows:

$$x_0 = (q_1, A),$$

and, for $0 \le r < m$ and $0 \le t < k$,

$$x_{1+rk+t} = (q_{\pi(r)}, M_t).$$

Thus

$$\{x_1, \ldots, x_{\ell-1}\} = Q \times \{M_0, \ldots, M_{k-1}\},$$

while the initial mode (q_0, A) is not a slope mode. Note in particular that

$$r_0 = r_1 = q_1.$$

Let

$$T = X_{\ell-1} X_{\ell-2} \cdots X_1 X_0.$$

Because the top is rightmost, X_0 is the top of T.

4.2 Transitions

The initial state is q_0, the initial stack symbol is A, and every state is final. The transitions are:

Header.

$$\delta(q_0, A, 1) = (q_1, T).$$

The first input 1 therefore installs the fixed mode table and enters mode x_0.

Zero on counter symbols. For every $0 \le i < m$,

$$\delta(q_i, A, 0) = (q_i, \varepsilon).$$

Selector transitions. For $0 \le s < \ell - 1$,

$$\delta(r_s, X_s, 0) = (r_{s+1}, \varepsilon),$$

and

$$\delta(r_{\ell-1}, X_{\ell-1}, 0) = (q_0, \varepsilon).$$

At $s = 0$ this agrees with the generic counter-pop rule, because $x_0 = (q_1, A)$ and $r_1 = q_1$.

Slope transitions. For every $0 \le s < \ell$,

$$\boxed{\delta(r_s, X_s, 1) = (r_s, A^{s+1}X_s).}$$

Thus one 1 read in slope mode x_s inserts $s+1$ copies of A immediately below the visible top symbol X_s, so the mode remains x_s.

All unspecified transitions are undefined. There are no ε-moves.

Proposition 4.1 (Realtime upper construction)**.** *$U_{m,k}$ is a deterministic realtime pushdown automaton with exactly m states and $k+1$ stack symbols.*

Proof. There are no ε-transitions. The slope modes are pairwise distinct, so their 1- and selector 0-rules do not conflict. The only apparent overlap is $(q_1, A, 0)$ at x_0; the selector rule there is exactly the generic rule $(q_1, A, 0) \mapsto (q_1, \varepsilon)$. The header uses $(q_0, A, 1)$, which is not a slope mode. Hence the transition function is deterministic. The cardinalities follow from the definitions of Q and Γ. □

Definition 4.2 (Binary witness language)**.** For $m \ge 2$ and $k \ge 1$, define

$$\boxed{B_{m,k} = L(U_{m,k}).}$$

The full language may contain words outside the simple block pattern used below. The lower bound needs only an exact slice of $B_{m,k}$, which we now isolate.

4.3 Exact pure-zero completion slice

For $0 \le s < \ell$ define

$$\boxed{N_s(n) = \ell - s + (s+1)n.}$$

Lemma 4.3 (Exact pure-zero slice)**.** *For every $0 \le s < \ell$ and $n, r \ge 0$,*

$$\boxed{10^s 1^n 0^r \in B_{m,k} \quad \Longleftrightarrow \quad r = N_s(n) = \ell - s + (s+1)n.}$$

Proof. After the initial 1, the stack is T, the state is $r_0 = q_1$, and the top is X_0. Each of the next s zeros pops the current table symbol X_j and changes the state to r_{j+1}. Hence after 10^s the state/top mode is $x_s = (r_s, X_s)$ and the remaining table has exactly $\ell - s$ symbols.

Each of the next n ones applies the slope rule at x_s and inserts $s+1$ copies of A below X_s. Therefore the stack height becomes

$$\ell - s + (s+1)n.$$

Now consider an all-zero suffix. The first zero pops X_s and moves to the state required for the next table symbol. Any inserted A symbols are then popped one by one without changing that state. When they are exhausted, the next table symbol is exposed in precisely its designated state, and the selector process continues. Thus every subsequent zero removes exactly one stack symbol until the stack becomes empty. Since all states are final, acceptance occurs exactly when the zero suffix has length equal to the current stack height. A shorter suffix leaves a nonempty stack; a longer suffix encounters the empty stack before the input is exhausted. □

Corollary 4.4 (Distinct affine completion slopes)**.** *The $\ell = mk + 1$ prefix families*

$$p_s 1^n, \qquad p_s = 10^s, \qquad 0 \le s < \ell,$$

have pairwise distinct unique pure-zero completion slopes

$$w_s = s + 1 \in \{1, 2, \ldots, mk + 1\}.$$

Example 4.5 (The smallest new case)**.** Let $m = 2$ and $k = 1$. Then $\ell = 3$, $Q = \{q_0, q_1\}$, and $\Gamma = \{A, M_0\}$. The three slope modes are

$$x_0 = (q_1, A), \qquad x_1 = (q_1, M_0), \qquad x_2 = (q_0, M_0),$$

with slopes $1, 2, 3$. The exact slice is

$$1\,1^n 0^{3+n}, \qquad 10\,1^n 0^{2+2n}, \qquad 100\,1^n 0^{1+3n}.$$

The theorem below implies that no DPDA with two states and one stack symbol, even with ε-moves, can recognize the same language.

5 Configuration escape along a live slope ray

Let $M = (Q, \{0,1\}, \Gamma, \delta, q_0, Z_0, F)$ be an arbitrary DPDA, possibly with ε-moves, satisfying

$$L(M) = B_{m,k}.$$

Fix a slope index s. Consider the unique computation of M on the infinite input

$$p_s 1^\omega, \qquad p_s = 10^s.$$

Every finite prefix $p_s 1^n$ is live, because lemma 4.3 supplies the accepted continuation $0^{N_s(n)}$. Therefore the infinite computation cannot become trapped in an ε-divergence before consuming the next input symbol.

We call a position in this computation *raw* if it is any individual machine step boundary, including positions inside a sequence of ε-moves. At a raw position record the number of accumulation 1's consumed after p_s and the control-store pair (q, γ).

Raw continuation principle. Once a raw control-store configuration (q, γ) and an unread input suffix are fixed, determinism fixes the future computation. In particular, an enabled ε-move is forced and is independent of the next unread input symbol. Consequently a raw position reached after a prefix may be reused when the future input is changed, provided the already consumed prefix is unchanged. We use this elementary observation repeatedly below to transfer accepting continuations between equal raw configurations.

Lemma 5.1 (No raw configuration repetition)**.** *Along the computation on $p_s 1^\omega$, the same control-store configuration (q, γ) cannot occur at two distinct raw positions.*

Proof. Suppose first that the two occurrences have consumed different numbers $n < n'$ of accumulation ones. By lemma 4.3, the word

$$p_s 1^{n'} 0^{N_s(n')} \in B_{m,k}$$

is accepted. The accepting run reaches the same raw control-store configuration after the already consumed prefix $p_s 1^{n'}$: any ε-moves occurring at that point are forced and do not depend on whether the next unread symbol is 1 or 0. The earlier occurrence has the same state and the same full stack. By the raw continuation principle, supplying the same zero suffix therefore gives the same future computation, so M would also accept

$$p_s 1^n 0^{N_s(n')}.$$

By lemma 4.3, however, the unique pure-zero completion after $p_s 1^n$ has length $N_s(n)$, and $N_s(n') \neq N_s(n)$. Contradiction.

Suppose instead that the two occurrences have consumed the same amount of input. The segment between them contains only ε-moves. Since it starts and ends in the same state and full stack, determinism forces the same ε-cycle to repeat forever. Then no live continuation could ever be read, again a contradiction. □

Lemma 5.2 (Raw-height escape)**.** *Along the infinite computation on $p_s 1^\omega$, the raw stack height tends to infinity: for every H there is a raw position after which the stack height is always greater than H.*

Proof. For fixed H, only finitely many control-store configurations have stack height at most H, because both Q and Γ are finite. By lemma 5.1, each such configuration can occur at most once. Hence only finitely many raw positions have height at most H. □

6 Suffix-minimum steps and context-independent pumping

We now extract a pump that remains valid under an arbitrary lower stack context. This is the point at which ε-moves are absorbed into the argument rather than excluded.

Definition 6.1 (Step position)**.** A raw position t on an infinite computation is a *step* if its stack height is a suffix minimum:

$$h(t) \le h(t') \qquad \text{for every } t' \ge t.$$

Lemma 6.2 (Infinitely many steps)**.** *Every integer-valued height sequence tending to infinity has infinitely many step positions. In particular, each slope ray of M contains infinitely many steps.*

Proof. For every raw position u, the tail $\{h(t) : t \ge u\}$ has a minimum because $h(t) \to \infty$. Choosing a position at which that tail minimum is attained produces a step. As u tends to infinity, the corresponding step positions cannot remain in a finite set. □

Lemma 6.3 (Step-pump lemma)**.** *On every slope ray there exist a mode $[q, X]$, an integer $c \ge 1$, and a nonempty word $\rho \in \Gamma^+$ such that for every lower context $\eta \in \Gamma^*$,*

$$(q, \eta X) \xRightarrow{1^c} (q, \eta\rho X), \tag{2}$$

where the macro-computation may contain forced ε-moves, consumes exactly c input symbols 1, and never visits a stack height below $|\eta X|$.

Proof. By lemma 6.2, the ray has infinitely many steps, while there are only $|Q||\Gamma|$ possible modes. Choose two step positions $t_1 < t_2$ with the same mode $[q, X]$. Write the earlier stack as ηX.

Because t_1 is a step, the computation between t_1 and t_2 never drops below height $|\eta X|$. Since the stack top is the rightmost symbol, exposing any symbol of the lower prefix η would require first popping X and reaching height $|\eta| < |\eta X|$. Hence η is never exposed or modified. The later stack therefore has the form

$$\eta\rho X$$

for some $\rho \in \Gamma^*$.

We claim $\rho \neq \varepsilon$. If the two step heights were equal, then the untouched lower prefix η, together with the same final top symbol X and the same state q, would give exactly the same full control-store configuration at t_1 and t_2, contradicting lemma 5.1. Hence the later height is strictly larger and $\rho \in \Gamma^+$.

Let c be the number of input ones consumed between the two steps. Since the transition sequence never reaches below the initial height, every transition sees only the active suffix that begins with the original X and the symbols subsequently generated above the untouched lower context. No transition can inspect a symbol of η. Replacing η by any $\eta' \in \Gamma^*$ therefore reproduces the same deterministic sequence and yields

$$(q, \eta' X) \xRightarrow{1^c} (q, \eta' \rho X).$$

Finally, c cannot be zero. If $c = 0$, then (2) is a context-independent height-increasing ε-macro from mode $[q, X]$ back to itself. Applying the same deterministic macro repeatedly would produce an infinite ε-computation

$$\eta X \to \eta \rho X \to \eta \rho^2 X \to \cdots,$$

preventing the next live input symbol from ever being consumed. Thus $c \geq 1$. □

Definition 6.4 (Up-mode)**.** A mode $[q, X]$ satisfying lemma 6.3 for some $c \geq 1$ and $\rho \in \Gamma^+$ is called an *up-mode* of the corresponding slope ray.

The lemma immediately gives iteration:

$$(q, \eta X) \xRightarrow{1^{jc}} (q, \eta \rho^j X) \qquad (j \geq 0). \tag{3}$$

7 One up-mode cannot support two slopes

The next elementary observation is where the acceptance convention becomes useful.

Lemma 7.1 (Unary completion uniqueness)**.** *Fix any raw control-store configuration (p, γ) of a deterministic pushdown automaton in the model of section 3. There is at most one integer $r \geq 0$ such that starting from (p, γ) with input 0^r leads to acceptance.*

Proof. Assume 0^r and $0^{r'}$ are both accepted with $r < r'$. Determinism makes the two computations identical until the shorter input has consumed all r zeros, including every forced ε-move that is enabled independently of whether more input remains. The accepting computation on 0^r eventually reaches a final state with empty stack. The computation on $0^{r'}$ reaches the same empty-stack state while $r' - r > 0$ zeros remain unread. No transition is defined on the empty stack, so the longer word cannot be accepted. Contradiction. □

Lemma 7.2 (Slope separation)**.** *Let $s \neq t$. The slope rays $p_s 1^\omega$ and $p_t 1^\omega$ cannot have the same up-mode in a DPDA recognizing $B_{m,k}$.*

Proof. Suppose both rays use the same up-mode $[q, X]$. Take the context-independent pump from one occurrence:

$$(q, \eta X) \xRightarrow{1^c} (q, \eta \rho X), \qquad c \geq 1, \quad \rho \in \Gamma^+.$$

Because the pump is independent of the lower context, it applies at an occurrence of the same mode on each ray. Let those occurrences appear after n_s and n_t accumulation ones, with stacks

$$\gamma_s X, \qquad \gamma_t X.$$

After j pump iterations, (3) yields reachable raw configurations

$$(q, \gamma_s \rho^j X), \qquad (q, \gamma_t \rho^j X)$$

after respectively $n_s + jc$ and $n_t + jc$ accumulation ones.

By lemma 4.3, the corresponding prefixes have accepted pure-zero continuations of lengths

$$N_s(n_s + jc) \quad \text{and} \quad N_t(n_t + jc).$$

Run these two accepting zero computations in parallel. As long as the common upper segment $\rho^j X$ has not been completely removed, the two machines have the same state and the same visible active stack suffix above the fixed tails. Whenever a zero is consumed in one run before that boundary is exposed, a zero is also still available in the other run; otherwise the run with exhausted input would have a nonempty stack and could reach acceptance only through forced ε-moves, which the other run must take as well. Thus deterministic consuming and ε-steps remain synchronized up to the first exposure of the lower tails. In particular, the common segment is removed after the same number e_j of consumed zeros, and immediately after the transition that removes its final active symbol the two raw configurations are

$$(p_j, \gamma_s), \qquad (p_j, \gamma_t)$$

for the same state p_j.

This exposure occurs no later than acceptance in either run: an accepting computation must empty its whole stack, hence must first remove the common upper segment. The set of states is finite, so there is an infinite set $J \subseteq \mathbb{N}$ and one state p such that $p_j = p$ for every $j \in J$.

For $j \in J$, the remaining zero lengths

$$R_s(j) = N_s(n_s + jc) - e_j, \qquad R_t(j) = N_t(n_t + jc) - e_j$$

are accepted from the fixed raw configurations (p, γ_s) and (p, γ_t), respectively. By lemma 7.1, $R_s(j)$ is constant on J, and so is $R_t(j)$. Hence

$$N_s(n_s + jc) - N_t(n_t + jc)$$

is constant on J.

But from lemma 4.3,

$$\begin{aligned} N_s(n_s + jc) - N_t(n_t + jc) &= (\ell - s) + (s + 1)(n_s + jc) \\ &\quad - (\ell - t) - (t + 1)(n_t + jc) \\ &= ((s + 1) - (t + 1))cj + O(1) \\ &= (s - t)cj + O(1). \end{aligned}$$

Since $s \neq t$ and $c \geq 1$, this quantity is unbounded along the infinite set J, a contradiction. □

8 The product lower bound and hierarchy

We can now count modes.

Theorem 8.1 (Product lower bound)**.** *Let $m \geq 2$ and $k \geq 1$. If a deterministic pushdown automaton M in the general model of section 3, with arbitrary deterministic ε-moves allowed, recognizes $B_{m,k}$, then*

$$\boxed{|Q_M|\,|\Gamma_M| \geq mk + 1.}$$

Proof. There are $\ell = mk + 1$ slope rays $p_s 1^\omega$, one for each $0 \leq s < \ell$. By lemma 6.3, each ray has an up-mode. By lemma 7.2, different slope indices require different up-modes. Hence M has at least $\ell = mk + 1$ distinct nonempty state/top-symbol modes. Since the total number of such modes is at most $|Q_M||\Gamma_M|$, the claimed bound follows. □

Corollary 8.2 (Binary adjacent stack-alphabet hierarchy; resolution for $m \geq 2$)**.** *For every $m \geq 2$ and $k \geq 1$,*

$$\boxed{\mathcal{D}_{m,k} \subsetneq \mathcal{D}_{m,k+1}.}$$

The strictness is witnessed over the binary input alphabet.

Proof. By proposition 4.1, $B_{m,k}$ is recognized by a realtime DPDA with m states and $k+1$ stack symbols, so

$$B_{m,k} \in \mathcal{D}_{m,k+1}.$$

If $B_{m,k}$ belonged to $\mathcal{D}_{m,k}$, some recognizing DPDA would satisfy

$$|Q| \leq m, \qquad |\Gamma| \leq k,$$

and hence

$$|Q||\Gamma| \leq mk,$$

contradicting theorem 8.1. The non-strict inclusion is trivial by allowing an unused extra stack symbol. □

Relation to the open problem. Corollary 8.2 gives the strict inclusion asked for in Open Problem 3.2 for every $m \geq 2$ and $k \geq 1$, with a binary witness. In this precise sense it resolves the fixed-state stack-alphabet hierarchy question posed by Masopust for the general DPDA model, apart from the separately formulated stateless convention discussed above. The significance of theorem 8.1 is that the competing recognizer is a *general* DPDA: deterministic ε-moves are allowed throughout the lower-bound argument. Thus the proof addresses exactly the feature that was absent from the realtime hierarchy theorem.

Remark 8.3 (Stronger than the separation)**.** The hierarchy requires only the impossibility of the corner $(|Q|, |\Gamma|) = (m, k)$. Theorem 8.1 excludes the entire region

$$|Q||\Gamma| \leq mk.$$

Thus the witness gives a quantitative state–stack-symbol tradeoff, not merely a yes/no adjacent separation.

Remark 8.4 (ε-moves are given to the lower-bound adversary)**.** The upper witness is realtime, but the lower bound permits the competing machine to use arbitrary deterministic ε-moves. The role of lemmas 5.1 and 6.3 is precisely to prevent those moves from hiding a slope distinction in unbounded internal normalization. The theorem is therefore not a realtime lower bound in disguise.

9 Why the proof is representation-independent

It is useful to contrast theorem 8.1 with architecture-relative resource bounds. Suppose a particular DPDA design stores a semantic residual in a prescribed coordinate system: one residue class in finite control, a sign in a marker, and an unbounded magnitude in a homogeneous stack tail. One can then count how many such semantic coordinates fit into the available local observations. Such an argument may prove an exact lower bound *within that representation family*, but a different recognizer could encode the same residual using a different stack code, temporary ε-moves, or deeper store structure.

The present proof never assumes a residual coordinate system. The only information imported from the language is the accepted/rejected behavior of words of the form

$$10^s1^n0^r.$$

From that behavior alone, every recognizer is forced to expose $mk + 1$ distinct recurrent modes. The chain is

$$\begin{aligned} \text{right-continuation lengths} &\Longrightarrow \text{no configuration repetition} \\ &\Longrightarrow \text{context-independent up-pumps} \\ &\Longrightarrow \text{slope separation} \\ &\Longrightarrow |Q||\Gamma| \text{ lower bound.} \end{aligned}$$

This is why theorem 8.1 is an unrestricted recognition lower bound for the witness language.

10 Related work and scope of the result

Stack-alphabet hierarchies and the open problem. Masopust proved that a binary alphabet suffices for the stateless DPDA stack-alphabet hierarchy and extended the hierarchy to m-state realtime DPDAs for every $m \geq 1$ [1]. The same paper explicitly leaves the existence of the corresponding hierarchy for general m-state DPDAs open. To the best of our knowledge, no published resolution of that question is known; accordingly, we state the bibliographic claim narrowly and relative to Masopust's explicitly posed problem. It also explains why this is not an automatic extension: once ε-moves are available, internal stack work can collapse the separation mechanism used by the realtime witnesses. Open Problems 1.1 and 3.2 isolate this exact gap. The present theorem is designed for that setting; the adversarial recognizer may use ε-moves freely, while the witness itself remains realtime and binary.

State/stack-symbol tradeoffs. Goldstine, Price, and Wotschke studied transformations reducing the number of stack symbols in a PDA and the associated state blow-up [2]; broader descriptional-complexity surveys place states and pushdown symbols among mutually dependent resources [3]. Kiss and Rovan likewise study state and stack-symbol complexity in selected PDA subfamilies and emphasize that the two resources cannot in general be compressed into a single scalar complexity measure [4]. Such results motivate the tradeoff but do not by themselves yield the exact same-m adjacent hierarchy for deterministic ε-DPDAs established here. More recent work continues to study exact simultaneous state and stack-symbol costs in restricted pushdown models, for example input-driven pushdown automata [6].

Modes. The pair consisting of control state and top stack symbol is a natural local unit in deterministic pushdown analysis. Pighizzini uses the term *mode* in the study of unary DPDAs [5]. Our use is elementary: we count modes only after proving, through right-continuation slopes, that distinct rays cannot share one recurrent up-mode.

Relation to the author's preceding manuscripts. The motivating completion-aware 2:1 DPDA and its later rational-counting generalizations are analyzed in two companion manuscripts [7, 8]. Those works develop residuals, shortest completions, configuration ladders, and state–stack tradeoffs for explicit representations. The present paper should not be read as claiming that the binary hierarchy theorem follows formally from those machines. Rather, the earlier completion viewpoint suggested treating the length of a forced continuation as an externally visible measurement of hidden pushdown information. The hierarchy proof then abstracts that idea until the original counting representation is no longer needed.

Novelty boundary. The mathematical contribution claimed here is the explicit binary witness $B_{m,k}$, the ε-robust product lower bound $|Q||\Gamma| \geq mk + 1$, and the resulting fixed-state adjacent

hierarchy for the general DPDA model with $m \geq 2$. The broader ideas that states and stack symbols trade descriptional power, and that state/top pairs are useful local modes, are established in prior work. Our bibliographic claim is deliberately narrow: the theorem proved here answers the hierarchy question explicitly posed by Masopust in the stated model for $m \geq 2$. The contribution is the explicit binary witness, the ε-robust product lower bound, and the resulting strict hierarchy, rather than a claim that state–stack tradeoffs or mode arguments are new in themselves.

11 Discussion and open directions

11.1 What has been proved, and what has not

For clarity, the logical scope of the manuscript can be summarized in three layers. First, the upper construction is concrete and realtime: $U_{m,k}$ uses exactly m states, $k+1$ stack symbols, and a binary input alphabet. Second, the lower bound is unrestricted within the general model of section 3: an equivalent recognizer may use a completely different stack representation and arbitrary deterministic ε-moves, yet must satisfy $|Q||\Gamma| \geq mk+1$. Third, the resulting hierarchy statement is proved here for $m \geq 2$ under the single-initial-symbol general-model convention. The stateless $m = 1$ hierarchy is a prior result under Masopust's separately stated stateless convention.

What is *not* claimed is equally important. The paper does not claim that the particular upper machine $U_{m,k}$ is simultaneously optimal in states and stack symbols at every point of a Pareto frontier; the product theorem only excludes the region $|Q||\Gamma| \leq mk$. Nor does the proof establish a bounded-push hierarchy, because the header transition writes a fixed block whose length depends on m and k. Finally, we make no broader bibliographic priority claim beyond the documented relation to Masopust's open question.

11.2 An explicit semantic description of $B_{m,k}$

For the lower bound it is enough to define $B_{m,k}$ as the language of the explicit upper witness and isolate the exact slice of lemma 4.3. This is mathematically legitimate but not necessarily the most elegant presentation. A useful refinement would characterize the full language $L(U_{m,k})$ by a direct combinatorial rule. Such a description must account for words that leave the pure block form and later revisit slope modes during zero-driven table traversal.

11.3 Bounded push length

The header transition writes a fixed word of length $\ell = mk+1$ in one step. The general PDA model permits replacement by an arbitrary fixed word in Γ^*. If one imposed a bounded-push convention, compiling the header into shorter writes would generally require auxiliary control states or ε-steps and would change the exact resource accounting. Determining the sharp hierarchy under an additional push-length restriction is therefore a separate descriptional problem.

11.4 The stateless convention

Masopust's stateless hierarchy allows a nonempty initial pushdown word in the stateless formulation, whereas the general model used for the new theorem starts from one initial stack symbol. The known $m = 1$ hierarchy and the present $m \geq 2$ hierarchy therefore cover the intended phenomenon under their stated conventions, but a uniform theorem under one single initial-store convention would be a useful cleanup result.

11.5 Beyond adjacent hierarchy

The product theorem suggests asking for a fuller Pareto frontier. For the specific witness $B_{m,k}$, how close is the lower hyperbola

$$|Q||\Gamma| \geq mk + 1$$

to the exact set of achievable state/stack-symbol pairs? The explicit upper point $(m, k+1)$ is only one point on that frontier. Determining exact realizability for other factorizations would sharpen the resource tradeoff.

11.6 Returning to completion-aware counting languages

The proof technique was developed while trying to escape representation-scoped lower bounds in completion-aware counting DPDAs. A natural next question is whether completion slopes or related quotient measurements can prove unrestricted recognition lower bounds for specific rational counting languages $L_{p,q}$ or for the open state–alphabet frontiers identified in the preceding manuscripts. The hierarchy witness was deliberately engineered to make the slopes maximally separable; counting languages impose more arithmetic structure and may require a stronger invariant.

12 Conclusion

We constructed, for every $m \geq 2$ and $k \geq 1$, a binary language $B_{m,k}$ with an m-state, $(k+1)$-stack-symbol realtime DPDA and proved that every equivalent general DPDA satisfies

$$|Q||\Gamma| \geq mk + 1.$$

The resulting strict inclusion

$$\mathcal{D}_{m,k} \subsetneq \mathcal{D}_{m,k+1}$$

shows that deterministic ε-moves do not collapse the fixed-state stack-alphabet hierarchy. In the general single-initial-symbol model for $m \geq 2$, this resolves the binary strict-hierarchy question stated in Open Problems 1.1 and 3.2; the separately formulated stateless case is already known from Masopust's work.

The proof is driven by completion slopes rather than by a prescribed stack code. Distinct slopes prevent configuration repetition; raw-height escape creates context-independent up-pumps; and a shared pump would force two affine completion functions to differ by a constant and by a nonzero linear term simultaneously. This converts externally observable right-continuation behavior into an unrestricted state/top-mode lower bound.

The route from the original completion-aware counting DPDA to the binary hierarchy is therefore methodological rather than deductive. The concrete machines suggested that stack contents can be probed by the continuations they require. The present witness turns that observation into a general descriptional lower-bound mechanism in which the recognizer is free to choose any internal representation.

A A compact view of the witness

For reference, the binary witness construction can be summarized in the following table. Here $\ell = mk + 1$ and $x_s = (r_s, X_s)$.

Purpose	Source/input	Action
Header	$(q_0, A), 1$	write $T = X_{\ell-1} \cdots X_0$, enter q_1
Counter pop	$(q_i, A), 0$	pop A, stay in q_i
Selector	$(r_s, X_s), 0$, $s < \ell - 1$	pop X_s, enter r_{s+1}
Final selector	$(r_{\ell-1}, X_{\ell-1}), 0$	pop $X_{\ell-1}$, enter q_0
Slope s	$(r_s, X_s), 1$	replace X_s by $A^{s+1}X_s$

Table 3: Transition summary for $U_{m,k}$. All states are final and all unspecified transitions are undefined.

B Proof dependency map

The logical dependency of the lower bound is deliberately short:

$$\boxed{\text{exact slice}} \Rightarrow \boxed{\text{no raw repetition}} \Rightarrow \boxed{\text{raw-height escape}} \Rightarrow \boxed{\text{step pump}} \Rightarrow \boxed{\text{slope separation}} \Rightarrow \boxed{|Q||\Gamma| \geq mk + 1}.$$

The first box is a property of the explicit realtime upper witness. Every later box concerns an arbitrary equivalent DPDA with ε-moves allowed.

Statements and Declarations

Funding. No funding was received for conducting this study.

Competing interests. The author has no relevant financial or non-financial interests to disclose.

References

[1] T. Masopust. A note on limited pushdown alphabets in stateless deterministic pushdown automata. *International Journal of Foundations of Computer Science*, 24(3):319–328, 2013. DOI: 10.1142/S0129054113500068; arXiv:1208.5002.

[2] J. Goldstine, J. K. Price, and D. Wotschke. On reducing the number of stack symbols in a PDA. *Mathematical Systems Theory*, 26:313–326, 1993. DOI: 10.1007/BF01189852.

[3] J. Goldstine, M. Kappes, C. M. Kintala, H. Leung, A. Malcher, and D. Wotschke. Descriptional complexity of machines with limited resources. *Journal of Universal Computer Science*, 8(2):193–234, 2002. DOI: 10.3217/jucs-008-02-0193.

[4] L. Kiss and B. Rovan. Descriptional complexity of push down automata. In *Information Technologies – Applications and Theory (ITAT 2020)*, CEUR Workshop Proceedings, Vol. 2718, pp. 59–66, 2020.

[5] G. Pighizzini. Deterministic pushdown automata and unary languages. *International Journal of Foundations of Computer Science*, 20(4):629–645, 2009. arXiv:0905.1248.

[6] O. Martynova. Exact descriptional complexity of determinization of input-driven pushdown automata. In *Implementation and Application of Automata (CIAA 2024)*, pp. 249–260, 2024. arXiv:2404.10516.

[7] A. E. Bütün. Completion Frontiers and Quotient Geometry in Deterministic Pushdown Automata for Rational Binary Counting. Manuscript submitted for publication, 2026.

[8] A. E. Bütün. Exact Structure and Resource Bounds in Deterministic Pushdown Automata for Binary Counting Languages. Manuscript submitted for publication, 2026.